

AI-Based Academic Tutoring approaches: A Mini Systematic Literature Reviews

Therusha REDDY¹, Nosipho DLAMINI², Sandhya HARRINARAIN³, Keagile MOLATEDI⁴,
Bongiwe MACUACUA⁵

Durban University of Technology , 7 Ritson Road, Durban, 4001, South Africa

Tel: +27 31 373 2000 , Fax: +27 31 373 2175 , Email: ¹22406937@dut4life.ac.za,

²22411296@dut4life.ac.za, ³22458894@dut4life.ac.za, ⁴22433887@dut4life.ac.za,

⁵22419010@dut4life.ac.za

Abstract: Include a short abstract of about 100-150 words summarising the problem statement, aim of the work, research method applied, results and conclusion in one paragraph.. Use abstract style.

Keywords: Insert main key words.

1. Introduction

P1- Context. Current state of academic tutoring in higher education. The availability bottleneck caused by high student-to-tutor ratios. Mention of traditional on-campus support models and their limitations.

P2- brief description, review solution and mention aspects not able to be addressed.

P3 - explaining certain aspects, expand and explain why its important to use.

P4 This study seeks to address (limitation).... Methodology identifies problem, proposed solution, evaluate, etc. using metrics. Data will be gathered from..... Uses primary/secondary dataset. The study utilises these tools.... Mention languages and db. This study is guided by the following objectives...

1.1 Proposed Solution

1.2 Objectives

1. To conduct a literature on AI-based tutoring and identify the limitations.
2. To propose a solution for...
3. To design and develop...
4. To evaluate the performance of hybrid systems using accuracy, recall, precision and F1 score.

2. Literature Review

The main goal of this literature review is to provide a systematic thematic synthesis of research on AI-based academic tutoring systems. This section is guided by the four research questions established in the study's research methodology, which progress from technical foundations to educational outcomes. It emphasizes the descriptive mapping of themes throughout the final 24 chosen articles.

2.1 Existing AI approaches for Academic Tutoring

This literature reveals five main architectural approaches. These are LLMs, MIA systems, Hybrid approaches and rule-based systems, along with Machine Learning (ML) and RAG (standalone approaches).

LLM-Based systems are represented by 66% of the reviewed studies. These systems utilise models such as GPT-4, Llama-3, and

DeepSeek R1, which generate fluid, human-like explanations in real time[4], [12], [15]. These systems demonstrate high technical performance such as the Eliminate-Then-Select framework which achieved a 96.1% accuracy on ScienceQA benchmarks [12]. However, it carries serious risks, such as a 28% factual error rate and a 45% student over-reliance rate where learners simply copy answers without

active thinking [4], [9].

MIA appeared in approximately 38% of systems from the reviewed studies. This system was developed to split different teaching tasks like testing and lesson planning among cooperative specialized software agents. This system prioritises structure and accountability over conversational flexibility however, it still suffered from mechanical and robotic dialogue [5], [6].

Historically rule-based systems used strict, pre-programmed guidelines to provide tutoring responses. Although the system is highly reliable and safe, they could not adapt to unpredictable student questions [4].

The percentage overlap of 66% (LLM) and 37% (MIA/rule-based) indicates hybrid implementations that combine LLM flexibility with agent-based control structures [6], [20].

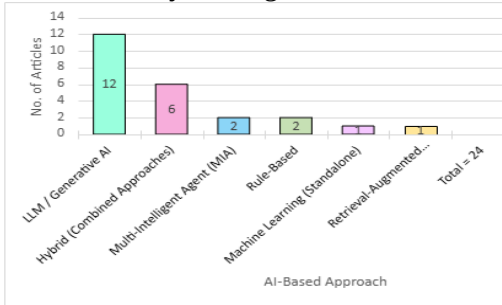


Figure 1: Various AI-Based Approaches Across 24 Articles

Figure 2 summarises the main AI-based tutoring approaches and outcomes identified across the 24 selected studies. LLM/GenAI appears in 66% of studies, while Rule/MIA appears in 38%. Notably, pedagogical scaffolding appears in only 33% of implementations, revealing a focus on technology over teaching. However, the success of all these systems depends heavily on the data they are trained on.

2.2. Theoretical Evaluation of approaches

The table below presents a comparative theoretical evaluation of the approaches mentioned previously in the literature. The evaluation examines each approach across six key metrics.

Table 1: Comparative Evaluation of AI Tutoring System Architectures

Approach	Accuracy	Precision	Factual Error Rat	Student over-reliance	Pedagogical Grounding	Response Adaptability
LLM Systems	✓	✓	✗	✗	✗	✓
MIA Systems	✓	✓	✓	✓	✗	✗
Rule-based Systems	✓	✓	✓	✓	✓	✗
Hybrid Systems	✓	✓	✓	✓	✗	✓

The evaluation reveals distinct competing differences in the architectural approaches. LLM systems achieve high technical performance metrics (96.1% accuracy, 0.995 precision) [12], [17]. It also introduces significant pedagogical risks including a 28% factual error rate and 45% student over-reliance [4], [9]. Furthermore, 68% of LLM implementations lack grounding in formal learning theories such as Constructivism or Bloom’s Taxonomy [18]. Rule-based systems offer reliability and strong pedagogical grounding through pre-programmed guidelines but cannot adapt to unpredictable student queries [4]. MIA systems provide structured task delegation and lower over-reliance risks but suffer from mechanical dialogue [5], [6]. Hybrid architectures emerge as the optimal theoretical balance, combining LLM conversational flexibility with MIA accountability and RAG integration for improved factual grounding, though their pedagogical grounding remains unproven [6], [8], [14].

2.3. Theoretical evaluation of Hybrid Systems

Based on the theoretical evaluation of the approaches, hybrid architectures emerge as the most promising approach for AI-based academic tutoring.

Table 2: In-Depth Evaluation of Hybrid System Sub-Approaches

Hybrid Sub-Approach	Accuracy	Precision	Factual Errors	Over-reliance	Pedagogical Grounding	Adaptability
RAG-Enhanced Systems	✓	✓	✓	✗	✗	✓
Agent-Orchestrated Systems	✓	✓	✓	✓	✗	✓
Scaffolding-Integrated Systems	✓	✓	✓	✓	✓	✓
Full-Stack Hybrid Systems	✓	✓	✓	✓	✓	✓

RAG-Enhanced Systems reduce hallucination rates below 28% by grounding responses in institutional datasets, however it remains vulnerable to over-reliance [9], [14]. Agent-Orchestrated Systems (CrewAI, LangGraph) distribute functions across specialised agents, improving accountability [6], [8]. Scaffolding-Integrated Systems incorporate heuristic questioning and hint generation, though only 33% of current implementations include these safeguards [15], [17]. Full-Stack Hybrid Systems combine all three approaches, offering the greatest potential for balancing LLM fluency with pedagogical effectiveness and reduced cognitive risks [6], [8], [14], [15], [17].

2.4. Technical tools

This section presents the technical infrastructure supporting AI-based tutoring systems, including programming languages, frameworks, databases, and deployment environments. The analysis examines how architectural choices influence system capabilities and limitations.

Component	LLM systems	MIA Systems	Rule-Based S	Hybrid Systems
Languages	Python, JavaScript	Python, Java, Prolog	Prolog, Python, Java	Python (LangChain, CrewAI)
LLM Backbones	GPT-4, Llama-3, DeepSeek	Smaller task-specific models	N/A	Multiple LLMs integrated
Databases	Vector(Pinecone, Weaviate, PostgreSQL)	SQLite, MongoDB, GraphDB	Relational	Vector+Relational + Graph
Deployment	Cloud (AWS, Azure, GCP)	Distributed/On-premise	On-Premise/Local	Hybrid cloud/on-premise
Frameworks	OpenAI API, Hugging Face	JADE, Jason, SPADE	Custom rule engines	LangGraph, CrewAI, RAG

Table 3: Technical Implementation of AI Tutoring Systems

LLM-based systems dominate in infrastructure complexity, requiring substantial computational resources (such as 8+ GPUs for 70B parameter models) and cloud deployment [12], [16]. Vector databases enable RAG implementation for improved factual grounding [14]. MIA systems utilise distributed agent frameworks (JADE, Jason) for task specialisation [6], [8]. Rule-based systems remain lightweight, operating on local servers with relational databases [4]. Hybrid implementations leverage orchestration frameworks (CrewAI, LangGraph) to coordinate multiple agents and LLMs, offering the most comprehensive technical architecture with improved transparency through traceable decision pathways [6], [8], [14].

This literature review examined AI-based academic tutoring systems across architectural approaches, theoretical evaluation, and technical implementation. Hybrid architectures combining LLM fluency with MIA accountability and RAG grounding emerge as the most effective approach, offering the optimal balance of technical performance (96.1% accuracy), reduced factual errors, and mitigated student over-reliance through structured scaffolding. LLM systems introduce significant pedagogical risks including 28% factual error rates and 45% student over-reliance, highlighting the necessity of hybrid designs that embed pedagogical controls.

3. Methodology

This study uses the Design Science Research Methodology (DSRM), as proposed by Peffers et al. [25].

This methodology is used to structure the design and evaluation of the proposed AI-based academic tutoring system.

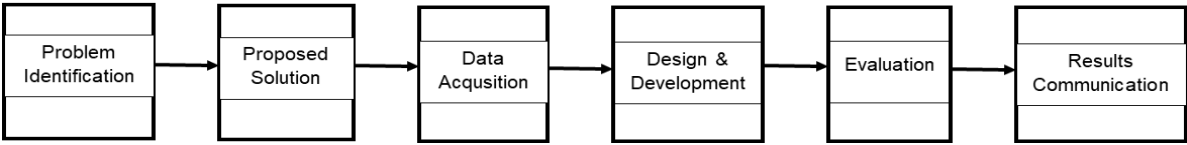


Figure 2. Design Science Research Methodology Diagram

The methodology consists of six phases which include problem identification, the proposed solution, data acquisition, design and development, evaluation and lastly results communication.

3.1 Proposed solution

The proposed system is a hybrid AI-based academic tutoring architecture integrating a LLM, RAG, MIA, and pedagogical scaffolding, to reduce student over-reliance (identified in 45% of reviewed studies) and pedagogical ungroundedness (identified in 68% of reviewed studies).

The LLM layer utilises local open-weight models, specifically DeepSeek-R1 and Qwen 3.5 running locally via Ollama, to generate explanations rather than immediate direct answers. The MIA layer divides backend operations into specialised, rule-based scripts. An Intent Agent intercepts and classifies the student's query to trigger cognitive forcing prompts. A RAG Agent retrieves relevant curriculum and a Guardrail Agent audits final outputs to block leaked complete answers. The RAG layer performs semantic search over the vector database, grounding responses in verified content. The Scaffolding layer uses a few-shot prompting to guide students through a Socratic progression which follows the order provided in the diagram below.

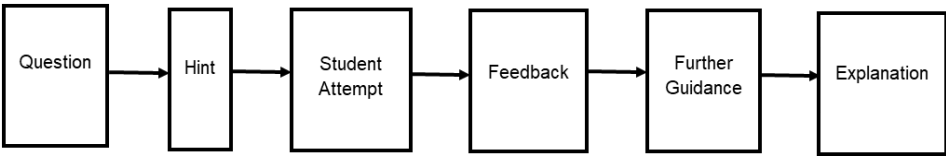


Figure 3. Socratic progression chain

Together, this integrated system operates as a secure, sequential pipeline that moves from Student Query, Intent Analysis ,Curriculum Retrieval, Scaffolded LLM Response, Guardrail Check and Output. to foster active cognitive engagement and mitigate the risk of student over-reliance.

3.2 Data Gathering

The primary dataset consists of institutional curriculum material such as lecture notes, course packs, tutorial sheets, and instructional guides covering first- to third-year content within the selected BINCT modules. This is restricted to material formally approved by the relevant faculty and department. Acquisition proceeds in five stages. First securing permission from the Head of Department and module lecturers, specifying which modules fall within scope. Secondly, obtaining the current approved version of each resource, avoiding outdated content. Thirdly, standardising collected material (PDF, DOCX, PPTX) into plain text. Fourth, dividing this text into semantically coherent chunks sized to balance specificity with retained context and fifth, embedding each chunk and storing it in a vector database with associated metadata, making it retrievable by the RAG Agent at query time. The table below presents a sample of the primary dataset that will be used.

Table 4: Dataset Sample

Course/Module Topic	Year/Level	Document Type	Content/Chunk (excerpt)	Source Reference
---------------------	------------	---------------	-------------------------	------------------

BICT101	Database Fundamentals	1st	Lecture Notes	"Relational databases organise data into tables with rows and columns..."	DB101_Module.pdf
BICT101	SQL Queries	1st	Tutorial	"SELECT statement retrieves data from database tables..."	SQL_Tutorial.pdf
BICT201	Data Structures	2nd	Lecture Notes	"Arrays store elements of same type in contiguous memory..."	DS201_Notes.pdf
BICT201	Algorithms	2nd	Tutorial	"Sorting algorithms arrange elements in specific order..."	Algo_Tutorial.pdf
BICT202	OOP Concepts	2nd	Course Pack	"Object-Oriented Programming uses classes and objects..."	OOP202_CP.pdf
BICT301	Software Engineering	3rd	Lecture Notes	"SDLC phases include requirements, design, implementation..."	SE301_Notes.pdf

This data is unstructured academic text. Each chunk carries six attributes such as Course/Module, Topic, Year/Level, Document Type, Content/Chunk, and Source Reference. The data is unlabelled and is used for similarity-based retrieval. It is not used to train a model.

3.3 Design and Development

The system follows a four-tier architecture, with each layer assigned a specific technology to ensure pedagogical control, data privacy, and maintainability. From the literature review findings, it was identified that hybrid systems are the most promising approach. This design combines LLM fluency with MIA accountability and RAG grounding. The table below outlines the technologies selected to implement each layer.

Table 5: Selected technologies per Tier

Tier	Technology	Purpose in This Study
Client Presentation	HTML5, Tailwind CSS, JavaScript	Builds responsive, accessible interface for students and lecturers, real-time updates
Orchestration	Python 3, Flask	Coordinates Intent, RAG, and Guardrail agents, manages API routes and session state
AI Inference	Ollama (DeepSeek-R1, Qwen 3.5)	Runs open-weight LLMs locally; avoids cloud costs, keeps student data on-premises
Knowledge Retrieval	Chroma / pgvector	Stores embedded curriculum chunks, performs semantic similarity search for RAG agent
Development Environment	Visual Studio Code	Provides integrated development with extensions and AI-assisted coding support

This combination was selected as it addresses the limitations identified in the literature. Across contemporary academic deployments, this specific combination; Flask-backend orchestration, locally hosted Ollama models, and a vector store for RAG forms the most practical and auditable foundation for enforcing Socratic tutoring rules without compromising institutional data sovereignty.

3.4 Evaluation

The system will be evaluated using standard classification metrics such as accuracy (proportion of correct responses), precision (correct positive predictions), recall (ability to find relevant responses), and F1-score (harmonic mean of precision and recall). These metrics will be calculated based on a test set of student queries with ground-truth answers provided by domain experts. The hybrid system will be compared against the baseline approaches such as standalone LLM and rule-based systems. We anticipate the hybrid system will achieve superior performance in pedagogical metrics (reduced over-reliance, improved factual grounding) while maintaining competitive technical performance. Results will be presented as performance graphs and discussed in Section 6.

4. Technology Description

This section explains the importance of each technology choice by looking at why it matters, how it helps address the problems identified in the literature, and what it will be used for in this study. System Architecture and Data Flow The architecture separates the system into a frontend interface, a MIA

orchestration layer, a local AI inference engine, and a vector database. Table 6 outlines the core tools and frameworks used to develop the system,

Table 6: Technology description table

Technology	Why it matters	How it helps	What it is used for
Flask (Python)	LLMs lack pedagogical control (45% over-reliance), agents needed to enforce Socratic rules	Lightweight API development, coordinates multiple agents in single codebase	Orchestrates Intent, RAG, and Guardrail agents, routes queries, manages sessions
Ollama (DeepSeek-R1, Qwen 3.5)	68% lack pedagogical grounding, cloud APIs raise privacy concerns	Runs LLMs locally; eliminates costs, keeps student data on-premises	Generates explanations, hints, and guiding questions during tutoring
Vector Database (Chroma/pgvector)	LLMs have 28% factual error rate from hallucination, responses need grounding	Enables RAG; stores embeddings, performs semantic similarity search	Stores curriculum chunks, retrieved by RAG Agent to ground responses in verified content
Frontend (HTML5, Tailwind, JS)	System must be accessible and responsive for adoption	Delivers responsive interface, WebSockets enable real-time updates	Provides student query interface and lecturer dashboard
VS Code	Efficient development requires robust support	Provides terminal, debugging, and AI-assisted extensions	Entire system developed, debugged, and tested within the environment

This stack forms a practical foundation for enforcing Socratic tutoring rules without compromising institutional data sovereignty. This makes the system scalable, secure, and pedagogically grounded. Each choice directly addresses a limitation identified in the literature review.

4. Developments

The lit review section said this and this is what we used. Describe the progress made in building the system and present any mockups or prototypes developed. Development has commenced/will commence. Describe mockups or wireframes created for the system interface (include a figure showing the mockup).

Explain the technical tools evaluated during development (such as, testing Ollama's performance, vector database selection, Flask routing tests). Discuss any challenges encountered during development and how they were addressed. State the current development status (such as, “The backend agent architecture has been implemented, and the frontend mockups have been designed”).

5. Results

Present the findings from evaluating the system's performance. Development has not yet commenced. This section outlines anticipated results. Comparative evaluation. Intended outcome. Mention the evaluation results using the metrics defined in Section 3.4 (Accuracy, Precision, Recall, F1-score). Show graphs or tables comparing the performance of the proposed hybrid system against baseline approaches (such as, standalone LLM, rule-based system). Discuss the results, what do the numbers tell us about the system's strengths and weaknesses?

Compare performance across different query types or subject areas. Mention any limitations observed during testing (such as, latency issues, certain topics where accuracy was lower).

6. Business Benefits

Although the system has not yet been developed, its design is expected to deliver the following institutional and operational benefits once implemented. Reduced tutor bottleneck. The practical value and impact the system would bring to institutions, educators, and students.

Support for diverse learning needs, improved student retention and outcomes, Cost savings from reduced need for additional tutoring staff; scalable AI solution that grows with student numbers. Improved graduation rates; better-prepared graduates for the workforce.

7. Conclusions

Summarise the study, reflect on the findings, and suggest future work. This paper reviewed 24 studies to identify the core weaknesses of AI-based tutoring over-reliance, hallucination, and weak pedagogical grounding and proposed a hybrid LLM–RAG–MIA architecture with pedagogical scaffolding to address them.

Future work includes full implementation, etc. Ultimately, AI should serve as a collaborative partner in knowledge construction rather than a substitute for critical thinking.

Acknowledgements

The authors acknowledge the use of content generated by Artificial Intelligence (AI) (including but not limited to text, figures, images, and code) in the paper entitled "Enter Paper Title" in the following ways:

<Brief Explanation of each way use of content generated by AI/Generative AI has been leveraged/incorporated in the paper>

References

- [1] B. Nayanakantha, M. A. P. P. Dayarathne, and R. L. W. Koggalage, "Enhancing SQL Query Learning Using Chatbot with NLP: A Methodical Review," in 2025 9th SLAAI International Conference on Artificial Intelligence (SLAAI-ICAI), Colombo, Sri Lanka: IEEE, Nov. 2025, pp. 1–6. doi: 10.1109/SLAAI-ICAI68534.2025.11318499.
- [2] T. Jackson, N. Mware, and H. Jiang, "WIP: Developing AI-Powered Chatbots as Learning Assistants: A Work-in-Progress Study on Enhancing Undergraduate Computer Science Education," in 2025 IEEE Frontiers in Education Conference (FIE), Nashville, TN, USA: IEEE, Nov. 2025, pp. 1–5. doi: 10.1109/FIE63693.2025.11328214.
- [3] S. O. Branisteanu, L. Alboiaie, and A. Panu, "Capturing the changing landscape of student code in an era defined by AI," *Procedia Comput. Sci.*, vol. 270, pp. 2336–2345, 2025, doi: 10.1016/j.procs.2025.09.355.
- [4] D. Fernandes et al., "AI makes you smarter but none the wiser: The disconnect between performance and metacognition," *Comput. Hum. Behav.*, vol. 175, p. 108779, Feb. 2026, doi: 10.1016/j.chb.2025.108779.
- [5] N. Pham, H. Pham Ngoc, and A. Nguyen-Duc, "Fairness for machine learning software in education: A systematic mapping study," *J. Syst. Softw.*, vol. 219, p. 112244, Jan. 2025, doi: 10.1016/j.jss.2024.112244.
- [6] H. Chniti, A. Hadyaoui, and L. Cheniti-Belcadhi, "Multi-Intelligent Agents and Generative AI in Learner Assessment: A Scoping Review of Research Trends, Challenges, and Hybrid Opportunities," *Procedia Comput. Sci.*, vol. 270, pp. 3340–3351, 2025, doi: 10.1016/j.procs.2025.09.459.
- [7] P. Jayakumar, P. Priyadharshini, M. S. Nisha, A. T. K. Savitha, and G. Hariharan, "AI-Powered English Language Learning Systems: A Framework for Personalized Education Using Natural Language Processing," in 2025 2nd International Conference on Artificial Intelligence and Knowledge Discovery in Concurrent Engineering (ICECONF), Chennai, India: IEEE, Oct. 2025, pp. 1–7. doi: 10.1109/ICECONF65644.2025.11379566.
- [8] H. Tebourbi, S. Nouzri, Y. Mualla, and A. Abbas-Turki, "Artificial Intelligence Agents for Personalized Adaptive Learning," *Procedia Comput. Sci.*, vol. 265, pp. 252–259, 2025, doi: 10.1016/j.procs.2025.07.179.
- [9] B. J. Corbett and J. M. Tangen, "AI tutors vs. tenacious myths: Evidence from personalised dialogue interventions in education," *Comput. Hum. Behav.*, vol. 175, p. 108828, Feb. 2026, doi: 10.1016/j.chb.2025.108828.
- [10] Y. Chen, K. K. Chai, J. Loo, R. Moosaei, and J. Obstfeld, "GenAI-Empowered Group-Based Authentic Assessment for Network Engineering Courses," in 2025 IEEE Global Engineering Education Conference (EDUCON), London, United Kingdom: IEEE, Apr. 2025, pp. 1–7. doi: 10.1109/EDUCON62633.2025.11016382.
- [11] M. Nguyen, R. Agarwal, A. Malik, and R. Pandey, "Generative AI: elevating knowledge acquisition and retention and recall through autonomy, interactivity and engagement," *J. Enterpr. Inf. Manag.*, vol. 38, no. 6, pp. 1692–1719, Dec. 2025, doi: 10.1108/JEIM-08-2024-0433.
- [12] M. Wang, Z. Li, X. Zhao, and Q. Guo, "Eliminate-Then-Select: A human-centric reasoning framework for educational question answering with LLMs," *Inf. Process. Manag.*, vol. 63, no. 2, p. 104422, Mar. 2026, doi: 10.1016/j.ipm.2025.104422.
- [13] J. Pasalkar, T. Gajbhiye, O. Othal, and S. Hanwate, "CodeBuddy – An AI-Powered Code Learning Buddy," in 2025 IEEE 9th International Conference on Information and Communication Technology (CICT), Chennai, India: IEEE, Dec. 2025, pp. 1–5. doi: 10.1109/CICT67193.2025.11399106.
- [14] S. P. Rana, M. Dey, P. Patel, J. Requena, and C. Fu, "Prompting and Rag Vs. Student Engagement and Comprehension in Educational Technology," in 2025 IEEE Global Engineering Education Conference (EDUCON), London, United Kingdom: IEEE, Apr. 2025, pp. 1–7. doi: 10.1109/EDUCON62633.2025.11016579.
- [15] G. Cheng, W. Wong, L. Luo, and M. Yu, "Integrating a Scaffolding-Based, LLM-Driven Chatbot into Programming Education: A University Case Study," in 2025 International Symposium on Educational Technology (ISET), Bangkok, Thailand: IEEE, Jul. 2025, pp. 1–5. doi: 10.1109/ISET65607.2025.00047.
- [16] F. Neha and D. Bhati, "DeepSeek Models in STEM Education: Capabilities, Applications, and Challenges," Mar. 14, 2025. doi: 10.36227/techrxiv.174198447.71647707/v1.
- [17] M. Chrysopoulos and L. Moussiades, "Enhancing High School Programming Education Through LLM-Based Hint Generation," in 2025 International Workshop on Artificial Intelligence and Education (WAIE), Yokohama, Japan: IEEE, Sep. 2025, pp. 437–443. doi: 10.1109/WAIE67422.2025.11381301.
- [18] H. Zhao, Y. Wu, Z. Lu, X. Yu, W. Miao, and L. Chen, "SageJavon: A scalable AI tutor for personalized programming learning," *Inf. Process. Manag.*, vol. 63, no. 5, p. 104605, Jul. 2026, doi: 10.1016/j.ipm.2025.104605.
- [19] T. C. Freitas, A. Costa Neto, M. J. V. Pereira, and P. R. Henriques, "Goliath and the Cognitive Load Theory," *J. Comput. Lang.*, vol. 85, p. 101372, Nov. 2025, doi: 10.1016/j.cola.2025.101372.
- [20] V.-P. Sorescu and D. Andone, "Developing Intelligent Tutors with Artificial Intelligence for Digital Education," in 2025 IEEE Digital Education and MOOCS Conference (DEMOcon), Boca Raton, FL, USA: IEEE, Oct. 2025, pp. 1–6. doi: 10.1109/DEMOcon65705.2025.11282632.

- [21] S. Guardado, R. Parveen, Z. Zhang, M. Rayhan, and N. Tripathi, "Students' Perceptions of the Use of LLMs in Requirements Engineering Education: A Cross-University Empirical Study," in 2025 IEEE 33rd International Requirements Engineering Conference (RE), Valencia, Spain: IEEE, Sep. 2025, pp. 130–141. doi: 10.1109/RE63999.2025.00022.
- [22] B. Ahmedova et al., "AI-Driven Stylistic Analysis of PLC Programming Content for Industrial Automation Education," in 2025 Second International Conference on Intelligent Technologies for Sustainable Electric and Communications Systems (iTech SECOM), Coimbatore, India: IEEE, Oct. 2025, pp. 1–6. doi: 10.1109/iTechSECOM64750.2025.11307199.
- [23] A. Gupta, S. Sawhney, A. Nanda, and S. Sharma, "Leveraging Custom GPTs to Enhance Teaching and Learning Quality in Engineering Education: A Case Study from a Teaching Learning Center in India," in 2025 IEEE International Conference on Teaching, Assessment, and Learning for Engineering (TALE), Macao: IEEE, Dec. 2025, pp. 1–8. doi: 10.1109/TALE66047.2025.11346731.
- [24] E. Trubljanin, E. Taruh, S. Čakić, T. Popović, and L. Filipović, "Transforming matrix problem solving with intelligent tutoring systems," in Proc. 24th Int.Symp. INFOTEH JAHORINA (INFOTEH), 2025, pp. 1–6.
- [25] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *J. Manage. Inf. Syst.*, vol. 24, no. 3, pp. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.